



Escuela
Politécnica
Superior

Extracción y Estructuración de Contenido Lingüístico en PDFs para la Preservación de Lenguas Mayas



Máster en Inteligencia Artificial

Trabajo Fin de Máster

Autor:

Elías Alfonso Carrasco Guerrero

Tutor:

Juan Antonio Pérez Ortiz

Setiembre 2025



Universitat d'Alacant
Universidad de Alicante

Digitalización, estructuración y preservación lingüística de gramáticas mayas minorizadas mediante inteligencia artificial multimodal

Automatización de la preservación de lenguas minorizadas: un enfoque basado en modelos de lenguaje y marcado XML

Autor

Elías Alfonso Carrasco Guerrero

Directores

Juan Antonio Pérez Ortiz

Departamento de Lenguajes y Sistemas Informáticos



MÁSTER EN INTELIGENCIA ARTIFICIAL



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

Alicante, 3 de junio de 2026

Resumen

La aplicación desarrollada para este Trabajo de Fin de Máster consiste en un software modular programado en Python que se ejecuta desde la terminal de comandos. Su función principal es automatizar la extracción de texto y la comprensión visual de páginas en formato PDF pertenecientes a gramáticas de lenguas mayas. El sistema inyecta ejemplos estructurados de entrada y salida (few-shot) junto a un prompt especializado directamente en la memoria de la API de Gemini, logrando que el modelo de Inteligencia Artificial entienda la disposición tipográfica original y devuelva la información totalmente organizada en un archivo XML con etiquetas semánticas personalizadas (como `¡morpheme.analysis!` o `¡syntactic.analysis!`). Posteriormente, el pipeline procesa este archivo XML para generar de forma automática un documento en LaTeX que se compila para reconstruir la obra original en un nuevo archivo PDF con un acabado editorial profesional. El objetivo último es transformar "papel digital" estático en un conjunto de datos estructurados de alta precisión (Gold Standard) que sirva como combustible indispensable para que, en el futuro, otras Inteligencias Artificiales puedan entrenarse y aprender idiomas minorizados de bajos recursos computacionales. Este Trabajo de Fin de Máster aborda la problemática de la digitalización y estructuración de gramáticas normativas de lenguas mayas minorizadas (como Kaqchikel, K'iche' y Mam), cuyos documentos originales presentan estructuras tipográficas complejas que dificultan el procesamiento mediante técnicas de OCR convencionales. Se propone y desarrolla un sistema automatizado basado en Modelos de Lenguaje de Gran Tamaño (LLMs) a través de la API de Gemini, integrado en un pipeline modular desarrollado en Python. Con este flujo se consigue mitigar la pérdida de información semántica y espacial inherente a los métodos planos tradicionales, salvaguardando recursos científicos de alto valor como las glosas interlineales. La metodología se fundamenta en la transformación de documentos PDF en datos estructurados bajo el estándar XML, garantizando la integridad jerárquica mediante la definición de un Documento de Definición de Tipo (DTD) y el desarrollo de módulos específicos de corrección estructural (`xml.corrector.py`). El núcleo de la investigación se centra en un estudio empírico sobre el parámetro de temperatura del modelo, evaluando su impacto en la precisión y estabilidad del sistema bajo una estricta economía de tokens impuesta por las limitaciones de cuota del entorno operativo. Los resultados experimentales demuestran la existencia de un "punto dulce" (sweet spot) en temperaturas de 0.1 y 0.25, donde se alcanzan precisiones superiores al 90

Palabras clave: Inteligencia Artificial, LLM, Gemini API, Lenguas Mayas, Digitalización, XML, Procesamiento de Lenguaje Natural, OCR Estructurado.

Motivación y justificación

La preservación de la diversidad lingüística constituye uno de los desafíos más críticos en la era de la información. Las lenguas minorizadas, en particular aquellas pertenecientes a la familia maya como el Kaqchikel, el K'iche, el Mam y el Kanjobal, representan un patrimonio cultural de valor incalculable. Sin embargo, su supervivencia no sólo depende de su uso oral, sino también de la disponibilidad y accesibilidad de recursos académicos y normativos en formatos digitales modernos. En este contexto, la Academia de Lenguas Mayas de Guatemala (ALMG) desempeña un papel crucial como entidad rectora en la promoción y estandarización de estos idiomas. A través de su repositorio institucional de documentos (disponible en <https://almg.org.gt/documentos/>), la ALMG ha realizado un esfuerzo significativo por poner a disposición del público gramáticas normativas y textos de referencia, tal y como se muestra en la figura 1. No obstante, gran parte de este conocimiento se encuentra consignado en archivos PDF que, si bien facilitan la difusión, actúan esencialmente como "papel digital": una representación estática de la información que carece de estructura semántica.

Por ejemplo, ya en marzo de 2025, tuvieron una actualización de la web en la que eliminaron el 70Esta limitación impide que los datos lingüísticos (ejemplos gramaticales, reglas sintácticas, morfología) puedan ser procesados automáticamente por herramientas computacionales, buscadores especializados o sistemas de aprendizaje profundo. La motivación técnica de este trabajo radica, por tanto, en la superación de las barreras impuestas por el reconocimiento óptico de caracteres (OCR) convencional sobre este corpus específico de la ALMG, para generar corpus con morfologías clasificadas para entrenamiento de modelos. Las gramáticas lingüísticas de estas lenguas presentan una complejidad tipográfica elevada, que incluye tablas anidadas, glosas interlineales y estructuras jerárquicas que los sistemas tradicionales suelen interpretar de manera errónea. Ante este escenario, surge la oportunidad de emplear Modelos de Lenguaje de Gran Tamaño (LLMs), para comprender la arquitectura lógica de estos documentos institucionales y transformarla en datos estructurados bajo el estándar XML. Este proyecto busca proporcionar una solución eficiente que reduzca la carga de trabajo manual en la digitalización del patrimonio de la ALMG, garantizando que la información sea interoperable, consultable y apta para la generación de documentación científica profesional mediante LaTeX.

Documentos Publicados



Según la Ley de la Academia de las Lenguas Mayas de Guatemala, establece que dentro de sus funciones está traducir y publicar, previo cumplimiento de las leyes de la materia, códigos, leyes, reglamentos y otros textos legales o de cualquier naturaleza que se juzgue necesario a los idiomas Mayas.

Además de crear, implementar e incentivar programas de publicaciones bilingües y monolingües, para promover el conocimiento y uso de los idiomas mayas para fortalecer los valores culturales.

Para lo anterior la Academia ha realizado varias publicaciones en diferentes idiomas Mayas, y algunas de ellas se presentan a continuación:

Para mayor información pueden ingresar a la página de [contacto](#)

AllSearch				
Comunidad Lingüística	Tipo de Documento	Nombre del Documento (Idioma Maya)	Nombre del Documento (Idioma Español)	Descarga
Achi	Texto	jik'ib'al Ub'ee Tz'iib'Maya'ch'a'teem Achi	Gramática _Normativa Idioma Maya Achi	
Akateka	Texto	ik'ti'ey Skuyb'anil	Fábulas	
Akateka	Texto	ik'ti'ey Skuyb'anil	Fábulas (2024)	
Akateka	Texto	xolilal Yelalil Ya'ley Ti'e Akateko	Gramática Descriptiva Akateka	
Akateka	Texto	Tz'ib'b'il qichmam Akateka	Literatura Maya Akateka	
Akateka	Texto	Sb'i K'ultajb'al yet Qichmam Akateko	Toponimias Maya Akateko	

Figura 1.: Web de la ALMG

Agradecimientos

*A mi familia por estar ahí en todo momento.
A mi pareja por ayudarme en todo lo que puede y más.
A mi madre por dedicar su vida a sus hijos.
A mi padre por animarme con mi carrera en la ciencia.
A mis compañeros del Santander por guiarme y enseñarme cómo desarrollar software.
Sin ellos no habría podido hacer este trabajo.*

Pues hemos nacido para colaborar, al igual que los pies, las manos, los párpados, las hileras de dientes, superiores e inferiores. Obrar, pues, como adversarios los unos de los otros es contrario a la naturaleza.

Marco Aurelio (Meditaciones)

Solo los que se quemaron con el fuego aprendieron a usarlo.

Andrés Pedreño Muñoz
Cofundador de Torre Juana OST

Índice general

Resumen	v
1. Introducción	1
1.1. Definición del problema	1
1.2. Alcance y limitaciones	3
1.2.1. Restricciones de propiedad intelectual y publicación de datos . . .	3
A. Anexo I. Esquema de base de datos	9
B. Anexo II. Código de la aplicación SIDManager	11
Lista de acrónimos	36

Índice de figuras

1.	Web de la ALMG	VIII
1.1.	Comparación OCR vs. Kaqchikel	2
A.1.	Esquema de la base de datos	9

Índice de tablas

1.1. Comparativa de disponibilidad de modelos 2025-2026	4
---	---

Índice de Listados

B.1. Kaqchikel XML generado por el LLM a partir de la imagen de la página original	11
B.2. Función getNotReceivedDocuments del controlador	14
B.3. Función getNotReceivedDocuments del servicio	15
B.4. Función listDocuments del controlador	15
B.5. Función getFilteredDocuments del servicio	15
B.6. Función findFilteredDocuments del repositorio	15
B.7. Función checkDates del servicio	16
B.8. Función getCreateClientUserView del controlador	17
B.9. Función createClientUser del controlador	17
B.10.Función saveClientUser del controlador	17
B.11.Función getConfirmation del controlador	18
B.12.Función confirmUser del servicio	19
B.13.Función getLoginView del controlador	19
B.14.Función getRequestUpdatePasswordView del controlador	20
B.15.Función requestUpdatePassword del controlador	20
B.16.Función sendUpdatePasswordUrl del servicio	20
B.17.Función getConfirmationUrl del servicio	21
B.18.Función getUpdatePasswordView del controlador	21
B.19.Función getUpdatePasswordView del servicio	22
B.20.Función updatePassword del controlador	22
B.21.Función updatePassword del servicio	23
B.22.Clase DocumentModel	23
B.23.Clase SecurityConfig	25
B.24.Clase CustomUserDetailsService	27
B.25.Clase CustomUserDetailsService modificado	27
B.26.Clase CustomAuthenticationFailureHandler	28
B.27.Función encryptMessage de Encryptor	28
B.28.Función decryptMessage de Encryptor	29
B.29.La función getDataSource de DataSourceConfig	29
B.30.El contenido html para mostrar la tabla de documentos	29
B.31.El contenido html para conectar los links de Bootstrap 5	30
B.32.El contenido html para enviar un mensaje al front-end con Thymeleaf	30
B.33.La clase Mail	31
B.34.La clase ResponseEnum	35
B.35.La clase LenguajeConfig	35

1. Introducción

1.1. Definición del problema

La digitalización de obras lingüísticas complejas se enfrenta a un obstáculo fundamental: la dicotomía entre la representación visual y la estructura lógica. El problema central se desglosa en los siguientes puntos críticos:

1. Naturaleza heterogénea de las fuentes (PDFs no procesados)

A diferencia de los documentos digitales nativos, el corpus de trabajo presenta una gran disparidad en su calidad técnica. Se ha identificado que un volumen significativo de los archivos PDF no son documentos con texto seleccionable y fondo limpio, sino fotografías directas de las páginas del libro físico. Estas fuentes carecen de cualquier pre-procesamiento de imagen o capa de texto subyacente, lo que implica: Ruido visual: Presencia de sombras, texturas del papel y falta de contraste entre la tipografía y el fondo. Inexistencia de metadatos de texto: La imposibilidad de extraer caracteres mediante scripts sencillos de lectura de PDF, obligando al sistema a realizar una interpretación visual completa de la imagen.

2. Insuficiencia del OCR convencional

Los sistemas de OCR tradicionales están diseñados para la extracción de texto lineal en documentos con una disposición de página sencilla como la parte izquierda de la figura 1.1. Sin embargo, la amalgama de una baja calidad de imagen junto con la arquitectura tipográfica y análisis sintácticos y morfológicos de las gramáticas como en la parte derecha de la figura 1.1, genera fallos sistemáticos en:

- Sistemas de numeración jerárquica:

Divisiones en secciones, tablas y disposiciones en vertical que suelen confundirse con el texto horizontal.

- Glosas y ejemplos multilingües:

La alternancia entre el castellano y la lengua maya, a menudo en bloques adyacentes, que el OCR estándar tiende a mezclar al no reconocer los límites visuales de las columnas o tablas.

- Estructuras morfológicas complejas:

Paradigmas verbales que, al ser extraídos de una "foto como texto plano, se convierten en caracteres inconexos.

- Estructuras sintácticas de subrayado:

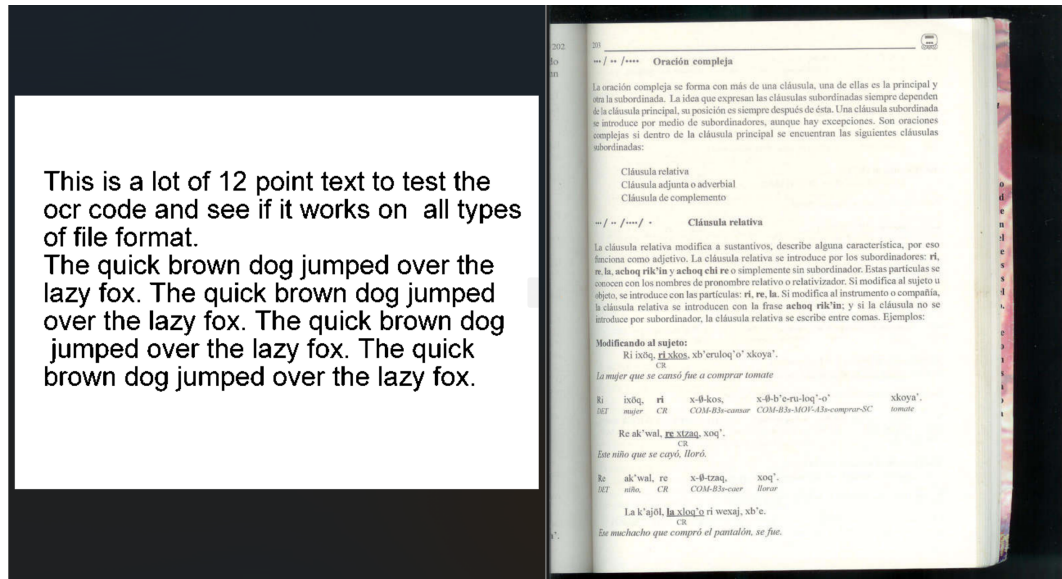


Figura 1.1.: Comparación OCR vs. Kaqchikel

Interpretaciones de segmentos de las oraciones subrayadas mediante análisis sintáctico.

- Texto sobrante:

a menudo al escanear un libro se recogen en la imágenes caracteres de las páginas adyacentes, manchas que dificultan la identificación de las letras, dibujos decorativos a inicio de página. . .

3. La pérdida de información al emplear PDF digitales

El formato PDF actúa aquí como un mero contenedor de imágenes. Al carecer de etiquetas semánticas (ej. `¡titulo!`, `¡ejemplo!`, `¡regla_gramatical!`), la información queda aislada de cualquier posibilidad de procesamiento masivo, análisis computacional o migración automática a formatos modernos. Se puede apreciar la diferencia entre ambos archivos de información comparando la figura (imagen) `pagina_original_Kaqchikel` en el código B.1.

4. El Desafío de la Inconsistencia Estructural en la IA

Incluso mediante el uso de LLMs multimodales, capaces de "ver" procesar estas fotos de libros, existe una tendencia a la generación de errores sintácticos en el marcado XML (etiquetas mal cerradas o jerarquías invertidas), especialmente en páginas de alta densidad informativa como la páginas con análisis sintácticos o morfológicos. Parte de este problema se debe a la creatividad de los LLMs que puede generar alucinaciones incluso empleando la metodología esquema de respuesta (response schema).

1.2. Alcance y limitaciones

El desarrollo de este Trabajo de Fin de Máster está condicionado por factores de carácter ético, legal y técnico que delimitan el marco de actuación. A continuación, se detallan las restricciones que han definido el alcance final de la investigación.

1.2.1. Restricciones de propiedad intelectual y publicación de datos

El corpus bibliográfico utilizado en este proyecto, consistente en gramáticas normativas de lenguas mayas, es propiedad intelectual de la Academia de Lenguas Mayas de Guatemala (ALMG). Como resultado de esta titularidad, se han establecido las siguientes limitaciones en cuanto a la difusión de datos:

- Prohibición de redistribución: No se permite la subida de los archivos PDF originales a repositorios públicos como GitHub. El acceso a las fuentes debe realizarse exclusivamente a través del portal institucional de la ALMG.
- Publicación de resultados parciales: El alcance de la publicación de resultados se limita a la estructura XML generada y a los fragmentos utilizados en el conjunto de pruebas (test set), preservando la integridad de la obra original.
- Lógica de replicabilidad: Para solventar esta limitación, el proyecto se enfoca en la liberación del código fuente (scripts de procesamiento), permitiendo que otros investigadores con acceso legítimo a los documentos puedan ejecutar el pipeline de digitalización de forma independiente.

Cabe destacar que el 70 % de los documentos mayas publicados en 2025 ya no están disponibles porque se han eliminado del catálogo de la web de la ALMG sin previo aviso. Esto provocó una gran dificultad en el trabajo ya que se tuvo que proseguir con los pocos documentos que se habían guardado en local antes de la actualización del catálogo.

Acotación de la carga de trabajo

De acuerdo con las directrices marcadas por la tutoría académica y con el fin de ajustar el proyecto a una carga de trabajo estrictamente acotada de 250 horas, se ha definido un alcance modular y selectivo:

- Selección de test set: En lugar de proceder a la digitalización de los libros completos, lo cual excedería el tiempo asignado en el cronograma, se ha optado por un análisis empírico profundo sobre una muestra representativa. Se han seleccionado un rango de 2 a 3 páginas de alta complejidad por cada idioma (Kaqchikel, K'iche, Mam y Qanjobal) para conformar el conjunto de validación final (Ground Truth).
- Priorización técnica: Se ha priorizado el perfeccionamiento de los módulos de validación estructural (`xml_corrector.py`) y la traducción automatizada a LaTeX sobre el procesamiento masivo de volumen, garantizando la robustez de la arquitectura antes de su escalabilidad futura.

Limitaciones técnicas de la API de Gemini

La dependencia de servicios de inteligencia artificial en la nube ha supuesto uno de los mayores desafíos logísticos del proyecto, debido a cambios imprevistos en las políticas de uso de Google Cloud y AI Studio, así como a la obsolescencia programada de sus arquitecturas:

- Restricción drástica de cuotas de uso: Durante el transcurso del desarrollo, el sistema sufrió una reducción crítica en los límites de peticiones permitidas. Se pasó de un margen operativo de 1.500 usos diarios (Requests Per Day - RPD) a un límite restrictivo de apenas 20 peticiones al día para los modelos en el nivel gratuito tal y como se ve en la tabla 1.1. Esta limitación del 98.6 % en la capacidad de cómputo obligó a reestructurar el flujo de trabajo, priorizando ejecuciones de prueba extremadamente precisas y limitando la posibilidad de realizar procesamiento masivos de páginas en una sola jornada.

Tabla 1.1.: Comparativa de disponibilidad de modelos 2025-2026

Modelos	Disponibilidad 2025 RPD	Disponibilidad 2026 RPD
gemini 1.5 flash	500	Deprecated
gemini 1.5 flash-lite	1500	Deprecated
gemini 2.0 flash	200	Deprecated
gemini 2.0 flash-lite	500	Deprecated
gemini 2.5 flash	20	20
gemini 2.5 flash-lite	20	20
gemma 3 27b	15.000	Deprecated
gemma 4 27b	No se había lanzado	Solo texto (al quitarle PDF)
gemini-3-flash-preview	No se había lanzado	20
gemini-3.1-flash-lite	No se había lanzado	500
gemini-3.5-flash	No se había lanzado	20

- Reducción en la disponibilidad y ciclo de vida de los modelos: Inicialmente, el entorno permitía el acceso concurrente a una variedad de entre 4 y 5 modelos de lenguaje distintos (incluyendo versiones experimentales y estables). No obstante, esta disponibilidad se vio mermada a solo 1 o 2 modelos activos por ventana de desarrollo debido a la depreciación masiva de arquitecturas estables. Específicamente, hitos de la plataforma como el del 29 de septiembre de 2025 supusieron la eliminación silenciosa de modelos de referencia como gemini-1.5-pro y gemini-1.5-flash, restringiendo la capacidad de realizar comparativas directas de rendimiento intramodelo de forma simultánea.
- Impacto en la metodología y presupuesto de tokens: Estas limitaciones técnicas externas impusieron una estricta economía de tokens y de tiempo. Se ha calculado matemáticamente el coste por llamada del pipeline mediante la ecuación de consumo estructural de tokens 1.3, considerando el prompt base y las capas de información visual y estructural que se envían a la API en cada iteración.

$$\text{Tokens por llamada} = 5,000 \text{ (prompt)} \tag{1.1}$$

$$+ [2,000 \text{ (imagen PDF)} + 2,000 \text{ (esquema XML)}] \times 2 \tag{1.2}$$

$$= 13,000 \text{ tokens} \tag{1.3}$$

Considerando las restricciones de la API para cuentas de desarrollo sin facturación asociadas a un techo de 250.000 tokens por minuto (Tokens Per Minute - TPM) y un límite de 10 solicitudes por minuto (RPM), la ventana de transmisión quedó restringida físicamente a un máximo de 19 llamadas simultáneas por minuto, forzando la inclusión de retrasos controlados en el script orquestador de 10 segundos en los reintentos. Sin embargo, tal y como se ha comentado anteriormente, la limitación más restrictiva fue la de peticiones por día (RPD - Request Per Day), ya que es más probable alcanzar 20 llamadas en un día que 19 llamadas en un minuto.

- Otra problemática de la API de gemini es que en momentos de alta demanda, esta no te devuelve resultados, te da error de servidor sobrecargado, lo que ha obligado a que en el proyecto se controle este flujo implementando un reintento a las 15 segundos.

Ante la inviabilidad operativa de las cuotas comerciales y los intentos fallidos de implementar arquitecturas locales basadas en la familia Gemma —debido a que gemma-3 fue completamente retirado por el proveedor y gemma-4 arrojó errores de servidor 500 al procesar de forma nativa la capa visual de los PDFs—, se optó estratégicamente por reorientar el pipeline hacia un modelo alternativo con límites específicos: gemini-3.1-flash-lite. Este cambio permitió eludir el bloqueo de cuota diaria (al ofrecer un techo de 500 peticiones), asumiendo una penalización marginal del 10% en la tasa de error de caracteres (CER) frente a los modelos de mayor escala, lo que obligó a robustecer los algoritmos heurísticos de post-procesamiento en el core del sistema.

Personalmente, este proyecto ha sido una oportunidad para profundizar en tecnologías y conceptos que no formaban parte de mi conocimiento previo. Desde el aprendizaje de nuevas herramientas como Spring Boot y Maven hasta la adopción de buenas prácticas como la documentación con Javadoc y el uso de estándares de Java, he adquirido habilidades que serán valiosas en futuros proyectos. Además, la experiencia de trabajar en un proyecto de esta envergadura ha reforzado mi capacidad para planificar, organizar y resolver problemas de manera eficiente.

En conclusión, este proyecto no solo ha cumplido con sus objetivos iniciales, sino que también ha sentado las bases para futuras mejoras y desarrollos. Espero que el trabajo realizado contribuya al avance de aplicaciones similares y sirva como referencia para otros desarrolladores e investigadores.

A. Anexo I. Esquema de base de datos

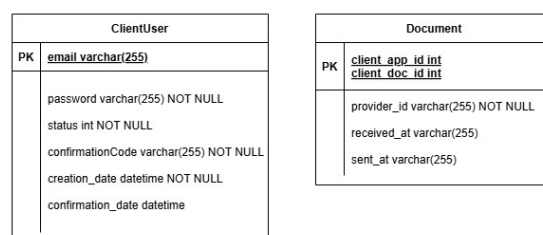


Figura A.1.: Esquema de la base de datos

B. Anexo II. Código de la aplicación SIDManager

Listado B.1: Kaqchikel XML generado por el LLM a partir de la imagen de la página original

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <document>
3   <page number="203">
4     <heading>... / .. / .... Oración compleja</heading>
5     <line>La oración compleja se forma con más de una cláusula
6       , una de ellas es la principal y</line>
7     <line>otra la subordinada. La idea que expresan las clá
8       usulas subordinadas siempre dependen</line>
9     <line>de la cláusula principal, su posición es siempre
10      después de ésta. Una cláusula subordinada</line>
11     <line>se introduce por medio de subordinadores, aunque hay
12      excepciones. Son oraciones</line>
13     <line>complejas si dentro de la cláusula principal se
14      encuentran las siguientes cláusulas</line>
15     <line>subordinadas:</line>
16
17     <line>Cláusula relativa</line>
18     <line>Cláusula adjunta o adverbial</line>
19     <line>Cláusula de complemento</line>
20
21     <heading>... / .. / .... / . Clausula relativa</heading>
22     <line>La clausula relativa modifica a sustantivos,
23      describe alguna característica, por eso</line>
24     <line>funciona como adjetivo. La clausula relativa se
25      introduce por los subordinadores: ri,</line>
26     <line>re, la, achoq rik'in y achoq chi re o simplemente
27      subordinador. Estas particulas se</line>
28     <line>conocen con los nombres de pronombre relativo o
29      relativizador. Si modifica al sujeto u</line>
30     <line>objeto, se introduce con las particulas: ri, re, la.
31      Si modifica al instrumento o compania,</line>
32     <line>la clausula relativa se introducen con la frase
33      achoq rik'in; y si la clausula no se</line>
34     <line>introduce por subordinador, la clausula relativa se
35      escribe entre comas. Ejemplos:</line>
```

```

26      <line>Modificando al sujeto:</line>
27      <interlinear_gloss>
28          <parallel_phrase>
29              <original>Ri ix\{"{o}q, ri xkos, xb'eruloq'o' xkoya
30                  '.</original>
31              <translation>La mujer que se cansó fue a comprar
32                  tomate</translation>
33          </parallel_phrase>
34          <morpheme_analysis>
35              <unit>
36                  <form>Ri</form>
37                  <gloss>DET</gloss>
38              </unit>
39              <unit>
40                  <form>ix\{"{o}q,</form>
41                  <gloss>mujer</gloss>
42              </unit>
43              <unit>
44                  <form>ri</form>
45                  <gloss>CR</gloss>
46              </unit>
47              <unit>
48                  <form>x-</form>
49                  <gloss>COM-</gloss>
50              </unit>
51              <unit>
52                  <form>\0-</form>
53                  <gloss>B3s-</gloss>
54              </unit>
55              <unit>
56                  <form>kos,</form>
57                  <gloss>cansar</gloss>
58              </unit>
59              <unit>
60                  <form>x-</form>
61                  <gloss>COM-</gloss>
62              </unit>
63              <unit>
64                  <form>\0-</form>
65                  <gloss>B3s-</gloss>
66              </unit>
67              <unit>
68                  <form>b'e-</form>
69                  <gloss>MOV-</gloss>
70              </unit>
71              <unit>
72                  <form>ru-</form>
73                  <gloss>A3s-</gloss>

```



```

73         </unit>
74         <unit>
75             <form>loq'-</form>
76             <gloss>comprar-</gloss>
77         </unit>
78         <unit>
79             <form>o'</form>
80             <gloss>SC</gloss>
81         </unit>
82         <unit>
83             <form>xkoya'.</form>
84             <gloss>tomate</gloss>
85         </unit>
86     </morpheme_analysis>
87     <syntactic_analysis>
88         <unit>
89             <form>ri xkos</form>
90             <gloss>CR</gloss>
91         </unit>
92     </syntactic_analysis>
93 </interlinear_gloss>
94
95 <interlinear_gloss>
96     <parallel_phrase>
97         <original>Re ak'wal, re xtzaq, xoq'.</original>
98         <translation>Este niño que se cayó, lloró.</
99             translation>
100     </parallel_phrase>
101     <morpheme_analysis>
102         <unit>
103             <form>Re</form>
104             <gloss>DET</gloss>
105         </unit>
106         <unit>
107             <form>ak'wal,</form>
108             <gloss>niño,</gloss>
109         </unit>
110         <unit>
111             <form>re</form>
112             <gloss>CR</gloss>
113         </unit>
114         <unit>
115             <form>x-</form>
116             <gloss>COM-</gloss>
117         </unit>
118         <unit>
119             <form>{\0}-</form>
120             <gloss>B3s-</gloss>

```

```

121         </unit>
122         <unit>
123             <form>tzaq,</form>
124             <gloss>caer</gloss>
125         </unit>
126         <unit>
127             <form>xq'</form>
128             <gloss>llorar</gloss>
129         </unit>
130     </morpheme_analysis>
131     <syntactic_analysis>
132         <unit>
133             <form>re xtzaq</form>
134             <gloss>CR</gloss>
135         </unit>
136     </syntactic_analysis>
137 </interlinear_gloss>
138
139 <interlinear_gloss>
140     <parallel_phrase>
141         <original>La k'aj\"{o}l, la xloq'o ri wexaj, xb'e
142         .</original>
143         <translation>Ese muchacho que compr\'o el pantal\'
144         on, se fue.</translation>
145     </parallel_phrase>
146     <syntactic_analysis>
147         <unit>
148             <form>la xloq'o ri wexaj</form>
149             <gloss>CR</gloss>
150         </unit>
151     </syntactic_analysis>
152 </interlinear_gloss>
153 </page>
154 </document>
155 }

```

Listado B.2: Función getNotReceivedDocuments del controlador

```

1 @GetMapping ("/not-received")
2 public String getNotReceivedDocuments(Model model){
3     logger.debug(" ----- Entering getNotReceivedDocuments ----- "
4     );
5     model.addAttribute("documents" , documentService.
6     getNotReceivedDocuments());
7     model.addAttribute ("isShowingUnsaveds", "Yes");
8     logger.debug(" ----- Exiting getNotReceivedDocuments ----- ");
9     return "documentsView";
10 }

```

Listado B.3: Función getNotReceivedDocuments del servicio

```
1 public List<Document> getNotReceivedDocuments() {  
2     logger.debug(" Entering getNotReceivedDocuments ");  
3     return documentRepository.findByReceivedAtIsNull();  
4 }
```

Listado B.4: Función listDocuments del controlador

```
1 @GetMapping("/list")  
2 public String listDocuments(  
3     @RequestParam(value = "min-date", required = false)  
4     String startDate,  
5     @RequestParam(value = "max-date", required = false) String  
6     endDate,  
7     Model model) throws Exception {  
8     logger.debug(" ----- Entering listDocuments ----- ");  
9  
10    DateRange dateRange = documentService.checkDates(startDate,  
11    endDate, model);  
12  
13    model.addAttribute("documents",  
14        documentService.getFilteredDocuments(  
15            dateRange.getStartDate(),  
16            dateRange.getEndDate()  
17        ));  
18    model.addAttribute("startDate", dateRange.getStartDate());  
19    model.addAttribute("endDate", dateRange.getEndDate());  
20    model.addAttribute("isShowingUnsaveds", "No");  
21  
22    logger.debug(" ----- Exiting listDocuments ----- ");  
23    logger.debug(" ----- ");  
24  
25    return "documentsView";  
26 }
```

Listado B.5: Función getFilteredDocuments del servicio

```
1 public List<Document> getFilteredDocuments(Date startDate, Date  
2     endDate) {  
3     logger.debug(" Entering getFilteredDocuments ");  
4     return documentRepository.findFilteredDocuments(startDate,  
5     endDate);  
6 }
```

Listado B.6: Función findFilteredDocuments del repositorio

```

1  @Query(value = "SELECT CLIENT_APP_ID, CLIENT_DOC_ID, PROVIDER_ID,
2    SENT_AT, RECEIVED_AT"
3    + " FROM documents WHERE SENT_AT > :startDate AND
4    SENT_AT < :endDate", nativeQuery = true)
5  List<Document> findFilteredDocuments(@Param("startDate") Date
6    startDate, @Param("endDate") Date endDate);

```

Listado B.7: Función checkDates del servicio

```

1  public DateRange checkDates(String stringStartDate, String
2    stringEndDate, Model model) {
3    final int daysRange = 10;
4    DateRange dateRange;
5    boolean isCorrectRange = true;
6
7    try {
8      if (DateUtils.checkDate(stringStartDate) && DateUtils.
9        checkDate(stringEndDate)) {
10        Date startDate = DateUtils.stringToUtilsDate(
11          stringStartDate);
12        Date endDate = DateUtils.stringToUtilsDate(
13          stringEndDate);
14
15        int daysBetween = DateUtils.daysBetween(startDate,
16          endDate);
17
18        if(daysBetween != -1 && daysBetween <= daysRange) {
19          dateRange = new DateRange(startDate, endDate);
20          logger.info(" Returning FILTERED documents ");
21        } else {
22          dateRange = new DateRange(-daysRange);
23
24          logger.warn(" Exceded maximum day range ( " +
25            daysBetween +
26            " ) with: " +
27            daysRange);
28          logger.warn(" Returning last 10 days ");
29
30          isCorrectRange = false;
31        }
32      } else if (DateUtils.checkDate(stringEndDate)) {
33        dateRange = new DateRange(DateUtils.stringToUtilsDate(
34          stringEndDate), -daysRange);
35        isCorrectRange = false;
36      } else if (DateUtils.checkDate(stringStartDate)) {
37        dateRange = new DateRange(DateUtils.stringToUtilsDate(
38          stringStartDate), daysRange);
39        isCorrectRange = false;
40      } else {
41        dateRange = new DateRange(-daysRange);

```

```

35     }
36   } catch (ParseException e) {
37     logger.error(" ERROR in getFilteredDocuments: " + e);
38     dateRange = new DateRange(-daysRange);
39   }
40
41   if (!isCorrectRange) {
42     model.addAttribute("responseMessage", ResponseEnum.
43       LIST_WARNING_1.getMessageKey());
44     model.addAttribute("isAWarningMessage", true);
45   }
46   return dateRange;
47 }

```

Listado B.8: Función getCreateClientUserView del controlador

```

1 @GetMapping("/user/create")
2 public String getCreateClientUserView() {
3     logger.debug("Entering GET: /user/create");
4
5     return "createClientUserView";
6 }

```

Listado B.9: Función createClientUser del controlador

```

1 @PostMapping("/user/create")
2 public String createClientUser(Model model,
3                               @ModelAttribute ClientUser
4                               clientUser,
5                               @RequestParam("psw-repeat") String
6                               passwordRepeat
7                               ) throws Exception {
8
9     logger.debug(" - Entering clientUserService.createClientUser()
10      , from POST: /user/create");
11
12     ResponseEnum responseMessage = clientUserService.
13       createClientUser(clientUser, passwordRepeat);
14
15     model.addAttribute("responseMessage", responseMessage.
16       getMessageKey());
17     addColorToResponse(model, responseMessage);
18
19     return "createClientUserView";
20 }

```

Listado B.10: Función saveClientUser del controlador

```

1 @Transactional

```

```

2 public ClientUser saveClientUser(ClientUser clientUser) throws
   Exception {
3     try {
4         if (clientUser.getPassword() != null) {
5             logger.debug("-- Entering encryptMessage()");
6             clientUser.setPassword(encryptFacade(clientUser.
               getPassword()));
7
8             logger.debug("-- Entering randomAlphanumeric()");
9             clientUser.setConfirmationCode(RandomStringUtils.
               randomAlphanumeric(30));
10
11             clientUser.setStatus(0);
12             clientUser.setCreationDate(new Date());
13             clientUserRepository.save(clientUser);
14         } else {
15             throw new IllegalArgumentException("Password cannot
               be null");
16         }
17     } catch (Exception e) {
18         logger.error(" ERROR: Possible error in encryption,
               confirmation code generation "
19             + " or saveClientUser:" + e);
20         throw e;
21     }
22
23     return clientUser;
24 }

```

Listado B.11: Función getConfirmation del controlador

```

1 @GetMapping("/user/confirm")
2 public String getConfirmation(Model model,
3                               @RequestParam("email") String email,
4                               @RequestParam("confirmationCode")
5                               String confirmationCode) {
6
7     logger.debug("- Entering clientUserService.confirmUser(),
8         from GET: /user/confirm");
9     ResponseEnum responseMessage = clientUserService.confirmUser(
10         email, confirmationCode);
11     logger.debug("- Exiting clientUserService.confirmUser()");
12
13     model.addAttribute("responseMessage", responseMessage.
14         getMessageKey());
15     addColorToResponse(model, responseMessage);
16
17     return "confirmationUser";
18 }

```

Listado B.12: Función confirmUser del servicio

```
1 @Transactional
2 public ResponseEnum confirmUser(String email,
3                                 String confirmationCode
4                                 ) {
5     ResponseEnum responseMessage;
6     ClientUser clientUser = clientUserRepository.
7         findByEmailAndConfirmationCodeAndStatus(email,
8         confirmationCode, 0);
9
10    if (clientUser != null) {
11        responseMessage = ResponseEnum.CONFIRM_SUCCESS;
12        clientUser.setStatus(1);
13        clientUser.setConfirmationDate(new Date());
14        clientUserRepository.save(clientUser);
15    } else {
16        logger.debug(" User not found, not correct or already
17            confirmed. ");
18        responseMessage = ResponseEnum.CONFIRM_WARNING;
19    }
20
21    return responseMessage;
22 }
```

Listado B.13: Función getLoginView del controlador

```
1 @GetMapping("/user/login")
2 public String getLoginView(Model model,
3                             HttpServletRequest request
4                             ) throws Exception {
5     String serverUrl = SharedProperties.get("serverUrl");
6     String requestUpdatePasswordUrl = serverUrl + "user/
7         requestUpdatePassword";
8     String createClientUserUrl = serverUrl + "user/create";
9     String errorMessage = (String) request.getSession().
10         getAttribute("error");
11
12     model.addAttribute("requestUpdatePasswordUrl",
13         requestUpdatePasswordUrl);
14     model.addAttribute("createClientUserUrl", createClientUserUrl);
15
16     if (errorMessage != null) {
17         ResponseEnum responseMessage = ResponseEnum.
18             LOGIN_WARNING_1;
19         model.addAttribute("responseMessage", responseMessage.
20             getMessageKey());
21         addColorToResponse(model, responseMessage);
22     }
23 }
```

```

18      // Elimina el mensaje de error de la sesión
19      request.getSession().removeAttribute("error");
20  }
21
22  return "loginView";
23  }

```

Listado B.14: Función getRequestUpdatePasswordView del controlador

```

1  @GetMapping("/user/requestUpdatePassword")
2  public String getRequestUpdatePasswordView() {
3      return "requestUpdatePasswordView";
4  }

```

Listado B.15: Función requestUpdatePassword del controlador

```

1  @PostMapping("user/requestUpdatePassword")
2  public String requestUpdatePassword(Model model,
3      @RequestParam("email") String email
4      ) throws Exception {
5
6      ResponseEnum responseMessage = clientUserService.
          sendUpdatePasswordUrl(email);
7
8      model.addAttribute("responseMessage", responseMessage.
          getMessageKey());
9      addColorToResponse(model, responseMessage);
10
11     return "requestUpdatePasswordView";
12 }

```

Listado B.16: Función sendUpdatePasswordUrl del servicio

```

1  public ResponseEnum sendUpdatePasswordUrl(String email) throws
    Exception {
2      ResponseEnum responseMessage;
3      ClientUser clientUser = clientUserRepository.findByEmail(email
        );
4
5      if (clientUser != null && clientUser.getStatus() == 1) {
6          String path = "user/updatePassword?email=";
7          String urlUpdatePassword = this.getConfirmationUrl(
            clientUser, path);
8
9          logger.debug(" - Entering sendMail");
10         String subject = " Change the password of your SIDManager
            user. ";
11         String body = " Click on the next URL if you want to
            change your password: ";

```



```
12     Mail.sendMail(clientUser.getEmail(), urlUpdatePassword,
13         subject, body);
14     logger.debug(" - Exiting sendMail");
15     responseMessage = ResponseEnum.UPDATE_PSW_SUCCESS_1;
16 } else {
17     responseMessage = ResponseEnum.UPDATE_PSW_WARNING_1;
18     logger.info(" The user does not exist or is not confirmed.
19         ");
20 }
21 return responseMessage;
22 }
```

Listado B.17: Función getConfirmationUrl del servicio

```
1 private String getConfirmationUrl(ClientUser clientUser,
2     String path
3     ) throws Exception {
4     StringBuilder confirmationUrl = new StringBuilder();
5     String serverUrl = SharedProperties.get("serverUrl");
6
7     confirmationUrl.append(serverUrl);
8     confirmationUrl.append(path);
9     confirmationUrl.append(clientUser.getEmail());
10    confirmationUrl.append("&confirmationCode=");
11    confirmationUrl.append(clientUser.getConfirmationCode());
12
13    return confirmationUrl.toString();
14 }
```

Listado B.18: Función getUpdatePasswordView del controlador

```
1 @GetMapping("/user/updatePassword")
2 public String getUpdatePasswordView(Model model,
3     HttpSession session,
4     @RequestParam("email") String email,
5     @RequestParam("confirmationCode") String confirmationCode
6     ) throws Exception {
7     boolean shouldFormBeShown = false;
8     ResponseEnum responseMessage = clientUserService.
9         getUpdatePasswordView(email, confirmationCode);
10
11     if (responseMessage.getStatus() == ResponseEnum.
12         UPDATE_PSW_SUCCESS_2.getStatus()) {
13         shouldFormBeShown = true;
14     } else {
15         model.addAttribute("responseMessage", responseMessage.
16             getMessageKey());
17         addColorToResponse(model, responseMessage);
18     }
```

```

15     }
16
17     model.addAttribute("shouldFormBeShown", shouldFormBeShown);
18
19     //Save the email to send it later in the post
20     session.setAttribute("email", email);
21
22     return "updatePasswordView";
23 }

```

Listado B.19: Función getUpdatePasswordView del servicio

```

1 public ResponseEnum getUpdatePasswordView(String email,
2                                           String confirmationCode
3                                           ) {
4     ResponseEnum responseMessage;
5     ClientUser clientUser
6         = clientUserRepository.
7         findByEmailAndConfirmationCodeAndStatus(
8             email, confirmationCode, 1);
9
10    if (clientUser != null) {
11        responseMessage = ResponseEnum.UPDATE_PSW_SUCCESS_2;
12    } else {
13        responseMessage = ResponseEnum.UPDATE_PSW_WARNING_2;
14        logger.info(responseMessage.getMessageKey());
15    }
16
17    return responseMessage;
18 }

```

Listado B.20: Función updatePassword del controlador

```

1 @PostMapping("/user/updatePassword")
2 public String updatePassword( Model model,
3                               HttpSession session,
4                               @ModelAttribute ClientUser clientUser,
5                               @RequestParam("psw-repeat") String passwordRepeat
6                               ) throws Exception {
7     String loginUrl = SharedProperties.get("serverUrl") + "
8         user/login";
9     String email = (String) session.getAttribute("email");
10    boolean shouldFormBeShown = true;
11    ResponseEnum responseMessage = clientUserService.
12        updatePassword(email, clientUser, passwordRepeat);
13
14    if (responseMessage.getStatus() == ResponseEnum.
15        UPDATE_PSW_SUCCESS_3.getStatus()) {
16        shouldFormBeShown = false;
17        model.addAttribute("loginUrl", loginUrl);
18    }
19 }

```

```

15     }
16
17     model.addAttribute("responseMessage", responseMessage.
18         getMessageKey());
19     model.addAttribute("shouldFormBeShown", shouldFormBeShown)
20     ;
21     addColorToResponse(model, responseMessage);
22
23     return "updatePasswordView";
24 }

```

Listado B.21: Función updatePassword del servicio

```

1 public ResponseEnum updatePassword(String email,
2     ClientUser clientUser,
3     String passwordRepeat
4 ) throws Exception {
5     ResponseEnum responseMessage;
6     String password = clientUser.getPassword();
7
8     if (password != null && password.equals(
9         passwordRepeat)) {
10         // Check if the user exists in the
11         // database
12         ClientUser existingUser = this.
13             getClientUser(email);
14
15         if (existingUser != null) {
16             existingUser.setPassword(
17                 encryptFacade(password));
18             clientUserRepository.save(
19                 existingUser);
20
21             responseMessage = ResponseEnum.
22                 UPDATE_PSW_SUCCESS_3;
23         } else {
24             responseMessage = ResponseEnum.
25                 UPDATE_PSW_WARNING_4;
26         }
27     } else {
28         responseMessage = ResponseEnum.
29             UPDATE_PSW_WARNING_3;
30     }
31
32     return responseMessage;
33 }

```

Listado B.22: Clase DocumentModel

```

1 package com.isban.scf.sid.model;

```

```
2
3 import javax.persistence.Column;
4 import javax.persistence.Entity;
5 import javax.persistence.Id;
6 import javax.persistence.Table;
7
8 @Entity
9 @Table(name = "documents") // Nombre de la tabla en la base de
10 // datos
11 public class Document {
12     @Id
13     @Column(name = "CLIENT_DOC_ID")
14     private Long clientDocId;
15
16     @Column(name = "CLIENT_APP_ID")
17     private Long clientAppId;
18
19     @Column(name = "PROVIDER_ID")
20     private String providerId;
21
22     @Column(name = "SENT_AT")
23     private String sentAt;
24
25     @Column(name = "RECEIVED_AT")
26     private String receivedAt;
27
28     // Getters y setters
29     public Long getClientDocId() {
30         return clientDocId;
31     }
32
33     public void setClientDocId(Long clientDocId) {
34         this.clientDocId = clientDocId;
35     }
36
37     public Long getClientAppId() {
38         return clientAppId;
39     }
40
41     public void setClientAppId(Long clientAppId) {
42         this.clientAppId = clientAppId;
43     }
44
45     public String getProviderId() {
46         return providerId;
47     }
48
49     public void setProviderId(String providerId) {
50         this.providerId = providerId;
51     }
52 }
```

```
50 }
51
52 public String getSentAt() {
53     return sentAt;
54 }
55
56 public void setSentAt(String sentAt) {
57     this.sentAt = sentAt;
58 }
59
60 public String getReceivedAt() {
61     return receivedAt;
62 }
63
64 public void setReceivedAt(String receivedAt) {
65     this.receivedAt = receivedAt;
66 }
```

Listado B.23: Clase SecurityConfig

```
1 @Configuration
2 @EnableWebSecurity
3 public class SecurityConfig {
4
5     private final UserDetailsService userDetailsService;
6
7     public SecurityConfig(UserDetailsService userDetailsService) {
8         this.userDetailsService = userDetailsService;
9     }
10
11     @Bean
12     public SecurityFilterChain securityFilterChain(
13         HttpSecurity http) throws Exception {
14         http
15             .authorizeHttpRequests(authorizeRequests ->
16                 authorizeRequests
17                     .antMatchers("/user/login",
18                         "/user/create",
19                         "/user/confirm",
20                         "/user/requestUpdatePassword",
21                         "/user/updatePassword",
22                         "/user/getAll",
23                         "/user/deleteAll",
24                         "/home",
25                         "/css/**",
26                         "/img/**",
27                         "/js/**")
28                     .permitAll() // Permitir acceso sin
29                             autenticación
30             )
31     }
```

```

28         .anyRequest().authenticated() // Cualquier
           otra ruta requiere autenticación
29     )
30     .formLogin(formLogin ->
31         formLogin
32         .loginPage("/user/login") // Página de
           login personalizada
33         .defaultSuccessUrl("/list", true) //
           Redirigir al home después del login
           exitoso
34         .permitAll()
35         .failureHandler(
           authenticationFailureHandler())
36     )
37     .logout(logout ->
38         logout
39         .logoutUrl("/logout")
40         .logoutSuccessUrl("/user/login?logout") //
           Redirigir a login después del logout
41         .invalidateHttpSession(true)
42         .deleteCookies("JSESSIONID")
43         .permitAll()
44     )
45     .exceptionHandling(exceptionHandling ->
46         exceptionHandling.accessDeniedPage("/user/
           accessDenied")); // Manejo de accesos
           denegados
47
48     return http.build();
49 }
50
51 protected void configure(AuthenticationManagerBuilder auth)
52     throws Exception {
53     auth.userDetailsService(userDetailsService).
54         passwordEncoder(passwordEncoder());
55 }
56
57 @Bean
58 public PasswordEncoder passwordEncoder() {
59     return new BCryptPasswordEncoder();
60 }
61
62 @Bean
63 public AuthenticationFailureHandler
64     authenticationFailureHandler() {
65     return new CustomAuthenticationFailureHandler();
66 }
67 }

```

Listado B.24: Clase CustomUserDetailsService

```
1
2 @Service
3 public class CustomUserDetailsService implements
4     UserDetailsService {
5
6     private final ClientUserRepository clientUserRepository;
7
8     public CustomUserDetailsService(ClientUserRepository
9         userRepository) {
10         this.clientUserRepository = userRepository;
11     }
12
13     @Override
14     public UserDetails loadUserByUsername(String username) throws
15         UsernameNotFoundException {
16         ClientUser user = clientUserRepository.findByEmail(
17             username);
18
19         if (user == null) {
20             throw new UsernameNotFoundException("User not found");
21         }
22
23         CustomUserDetails customUserDetails = new
24             CustomUserDetails(user);
25
26         return customUserDetails;
27     }
28 }
```

Listado B.25: Clase CustomUserDetailsService modificado

```
1
2 @Service
3 public class CustomUserDetailsService implements
4     UserDetailsService {
5
6     private final ClientUserRepository clientUserRepository;
7
8     public CustomUserDetailsService(ClientUserRepository
9         userRepository) {
10         this.clientUserRepository = userRepository;
11     }
12
13     @Override
14     public UserDetails loadUserByUsername(String username) throws
15         UsernameNotFoundException {
16         ClientUser user = clientUserRepository.findByEmail(
17             username);
18     }
```

```

14
15         if (user == null) {
16             throw new UsernameNotFoundException("User not found");
17         }
18
19         CustomUserDetails customUserDetails = new
20             CustomUserDetails(user);
21
22         return customUserDetails;
23     }

```

Listado B.26: Clase CustomAuthenticationFailureHandler

```

1  @Component
2  public class CustomAuthenticationFailureHandler implements
    AuthenticationFailureHandler {
3
4      @Override
5      public void onAuthenticationFailure(HttpServletRequest request
        , HttpServletResponse response, AuthenticationException
        exception)
6          throws IOException, ServletException {
7          // Establece un mensaje de error en la sesión
8          request.getSession().setAttribute("error", "Login error");
9
10         // Redirige de nuevo a la página de login
11         response.sendRedirect("/user/login");
12     }
13
14 }

```

Listado B.27: Función encryptMessage de Encryptor

```

1  public static String encryptMessage(String message) throws
    Exception {
2      if (message == null) {
3          throw new IllegalArgumentException("Message to be
            encrypted cannot be null");
4      }
5      byte[] key = SharedProperties.getEncryptKey().getBytes();
6
7      Cipher cipher = Cipher.getInstance(ALGORITHM);
8      SecretKeySpec secretKey = new SecretKeySpec(key, "AES");
9      cipher.init(Cipher.ENCRYPT_MODE, secretKey);
10     byte[] encryptedMessage = cipher.doFinal(message.getBytes());
11
12     return Base64.getEncoder().encodeToString(encryptedMessage);
13 }

```


Listado B.28: Función decryptMessage de Encryptor

```

1 public static String decryptMessage(String encryptedMessage)
2     throws Exception {
3     byte[] key = SharedProperties.getEncryptKey().getBytes();
4
5     byte[] messageBytes = Base64.getDecoder().decode(
6         encryptedMessage);
7     Cipher cipher = Cipher.getInstance(ALGORITHM);
8     SecretKeySpec secretKey = new SecretKeySpec(key, "AES");
9     cipher.init(Cipher.DECRYPT_MODE, secretKey);
10    byte[] decryptedBytes = cipher.doFinal(messageBytes);
11
12    return new String(decryptedBytes);
13 }

```

Listado B.29: La función getDataSource de DataSourceConfig

```

1 @Bean
2 @Primary
3 public DataSource getDataSource() {
4     logger.debug(" --- Creating driver for database connection ---");
5
6     DriverManagerDataSource dataSource = new
7         DriverManagerDataSource();
8     dataSource.setUrl(dbUrl);
9     dataSource.setUsername(dbUsername);
10
11     try {
12         String decryptedPassword = Encryptor.decryptMessage(
13             encryptedPassword);
14         dataSource.setPassword(decryptedPassword);
15     } catch (Exception e) {
16         logger.error(" ERROR while decrypting the database
17             password (check application.properties): ", e);
18         throw new RuntimeException();
19     }
20
21     logger.debug(" --- Driver was successfully created --- ");
22     return dataSource;
23 }

```

Listado B.30: El contenido html para mostrar la tabla de documentos

```

1 <table id="example" class="table table-striped">
2     <thead>
3         <tr>
4             <th colspan="5" id="tableHead"> Documents in the
5                 database </th>

```

```

5         </tr>
6         <tr>
7             <th id="columnHeader1"> Client app id </th>
8             <th id="columnHeader2"> Client doc id </th>
9             <th id="columnHeader3"> Provider id </th>
10            <th id="columnHeader4"> Sent at </th>
11            <th id="columnHeader5"> Received at </th>
12        </tr>
13    </thead>
14    <tbody>
15        <!-- Este bloque se repetirá para cada documento en la
16             lista -->
17        <tr th:each="document : ${documents}">
18            <td th:text="${document.clientAppId}"></td>
19            <td th:text="${document.clientDocId}"></td>
20            <td th:text="${document.providerId}"></td>
21            <td th:text="${document.sentAt}"></td>
22            <td th:text="${document.receivedAt ? : 'Not received
23                '}"></td>
24        </tr>
25    </tbody>
26</table>

```

Listado B.31: El contenido html para conectar los links de Bootstrap 5

```

1 <!-- CSS -->
2 <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/
3   libs/twitter-bootstrap/5.3.0/css/bootstrap.min.css">
4 <link rel="stylesheet" href="https://cdn.datatables.net/1.13.4/css
5   /dataTables.bootstrap5.min.css">
6
7 <!-- JavaScript -->
8 <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
9
10 <script src="https://cdnjs.cloudflare.com/ajax/libs/twitter-
11   bootstrap/5.3.0/js/bootstrap.bundle.min.js"></script>
12 <script src="https://cdn.datatables.net/1.13.4/js/jquery.
13   dataTable.min.js"></script>
14 <script src="https://cdn.datatables.net/1.13.4/js/dataTables.
15   bootstrap5.min.js"></script>

```

Listado B.32: El contenido html para enviar un mensaje al front-end con Thymeleaf

```

1 <!-- Show the message if it exists / paint it red if it is a
2   warning / print the one from messages.properties with that name
3   -->
4 <div id="errorMessageModal" class="modal" th:if="${responseMessage
5   }">
6     <div class="modal-content"
7       th:style="${isAWarningMessage ?

```

```

5         'background-color:red;' :
6         'background-color:green;'}">
7         <span class="close-button">&times;</span>
8         <p th:text="#{{responseMessage}}"></p>
9     </div>
10 </div>

```

Listado B.33: La clase Mail

```

1
2 public class Mail {
3     private static final Logger logger = LogManager.getLogger(
4         Mail.class);
5
6     /*
7     public void main(String [] args) throws Exception {
8         logger.debug(" - Entering sendMail");
9         String subject = "Confirm your sign up in SIDManager";
10        String body = "Click on the next URL to activate your
11        SIDManager user: ";
12        Mail.sendMail("daleaenter00@gmail.com", "urlConfirmation",
13        subject, body);
14    }
15    */
16
17    /**
18    * This method creates the session, it also specify the
19    server properties
20    * and set the message and its content (that includes the
21    receivers,
22    * the body of the mail...).
23    * Finally the method send the mail.
24    * @param confirmationCode
25    * @param receiver
26    *
27    * @throws Exception If a method exception occurred
28    */
29    public static void sendMail(String receiver,
30        String urlConfirmation,
31        String subject,
32        String body) throws Exception {
33        logger.debug("Entering Mail");
34
35        final String sender = SharedProperties.get( "mailSender" )
36            ;
37
38        final String password = SharedProperties.get( "
39            mailPassword" );
40        // Crear sesión de correo
41        Session session = getSession(sender, password);

```

```

35
36 // Get session logging in with server properties
37 //Session session = Session.getInstance(
    getServerProperties());
38
39 try {
40     // Create email message
41     Message message = createMessage(session, sender,
42         receiver, urlConfirmation, subject, body);
43
44     // Send email
45     Transport.send(message);
46
47     logger.debug("The mail was sent successfully.");
48 } catch (MessagingException e) {
49     logger.error("There was an error sending the email
50         : " + e.getMessage());
51     logger.error("Status of the VARIABLES: "
52         + " | session: " + session
53         + " | sender: " + sender
54         + " | receiver: " + receiver
55         + " | urlConfirmation: " +
56             urlConfirmation
57         + " | subject: " + subject
58         + " | body: " + body
59     );
60     throw new Exception();
61 }
62
63 logger.debug("Exiting Mail");
64
65 /**
66  * This method define who is the sender and receiver,
67  * it also specify the mail subject, body and attachment.
68  *
69  * @param session A session for the logged sender in
70  * Session format.
71  * @param sender A sender which is going to send the email
72  * in string format.
73  * @param receiver A receiver of the email in String
74  * format.
75  * @param confirmationCode A code for confirm a user in
76  * String format.
77  *
78  * @throws AddressException If an error with the address
79  * occurs
80  * @throws MessagingException If an error with the message

```

```
occurs
75      *
76      * @return the email message in Message format
77      */
78  private static Message createMessage
79      (Session session, String sender, String receiver, String
80       urlConfirmation, String subject, String body)
81      throws AddressException, MessagingException {
82
83      logger.debug("Entering setMessageAndItsContent");
84
85      Message message = new MimeMessage(session);
86
87      // Define who the sender is
88      message.setFrom(new InternetAddress(sender));
89
90      message.setRecipient(Message.RecipientType.TO, new
91                          InternetAddress(receiver));
92
93      // Set email subject
94      //message.setSubject("Confirm your sign up in SIDManager")
95      ;
96      message.setSubject(subject);
97
98      // Email body
99      //String content = "Click on the next URL to activate your
100      SIDManager user: " + urlConfirmation;
101      String content = body + urlConfirmation;
102
103      MimeBodyPart mimeBodyPart = new MimeBodyPart();
104      mimeBodyPart.setContent(content, "text/html; charset=utf-8
105      ");
106
107      // Create multipart email, mix the attach to the
108      email body.
109      MimeMultipart multipart = new MimeMultipart();
110      multipart.addBodyPart(mimeBodyPart);
111
112      message.setContent(multipart);
113
114      logger.debug("Exiting setMessageAndItsContent");
115
116      return message;
117  }
118
119  /**
120   * This method get the session of the sender with
```

```

        authentication.
117     *
118     * @param sender A sender which is going to send the email
        in string format.
119     * @param password The password of the sender email.
120     *
121     * @throws Exception If an error with the session occurs
122     *
123     * @return the session in Session format
124     */
125     private static Session getSession (final String sender,
        final String password) throws Exception {
126
127     // Set mail server settings
128     Properties props = getServerProperties();
129
130         // Get session logging in with user and password
131         Session session = Session.getInstance(props, new
            Authenticator() {
132             protected PasswordAuthentication
                getPasswordAuthentication() {
133                 return new PasswordAuthentication(sender,
                    password);
134             }
135         });
136         return session;
137     }
138
139
140     /**
141     * This method get the properties needed for the session (
        port, host...).
142     *
143     * @return the properties needed for the session (port and
        host).
144     *
145     * @exception Exception If an error with the properties
        occurs
146     */
147     private static Properties getServerProperties() throws
        Exception {
148         // Get the port and host
149         String host = SharedProperties.get("mailHost");
150         String port = SharedProperties.get("mailPort");
151
152         // Server properties are specified here
153         Properties props = new Properties();
154         props.put("mail.smtp.host", host);
155         props.put("mail.smtp.port", port);

```

```

156     props.put("mail.smtp.auth", "true"); //Si se requiere
        autenticacion y password
157     props.put("mail.smtp.starttls.enable","true");
158
159     //props.put("mail.smtp.socketFactory.class", "javax.net.
        ssl.SSLSocketFactory");
160     //props.put("mail.smtp.socketFactory.fallback", "false");
161
162     return props;
163 }
164 }

```

Listado B.34: La clase ResponseEnum

```

1 public enum ResponseEnum {
2     SIGN_UP_SUCCESS("singup.success", 210),
3     CONFIRM_SUCCESS("confirm.success", 220),
4     LOGIN_ERROR_1("login.error.1", 430);
5
6     private String messageKey;
7     private int status;
8
9     private ResponseEnum(String messageKey, int status) {
10         this.messageKey = messageKey;
11         this.status = status;
12     }
13
14     public String getMessageKey() {
15         return messageKey;
16     }
17
18     public int getStatus() {
19         return status;
20     }
21 }

```

Listado B.35: La clase LenguajeConfig

```

1 @Configuration
2 public class LenguajeConfig implements WebMvcConfigurer {
3
4     @Bean
5     public SessionLocaleResolver localeResolver() {
6         SessionLocaleResolver slr = new SessionLocaleResolver();
7         slr.setDefaultLocale(Locale.US); // Idioma por defecto:
            inglés
8         return slr;
9     }
10
11     @Bean

```

```
12     public LocaleChangeInterceptor localeChangeInterceptor() {
13         LocaleChangeInterceptor lci = new LocaleChangeInterceptor
14             ();
15         lci.setParamName("lang"); // Parámetro para cambiar el
16             idioma
17         return lci;
18     }
19
20     @Override
21     public void addInterceptors(InterceptorRegistry registry) {
22         registry.addInterceptor(localeChangeInterceptor());
23     }
24 }
```